

To install playwright from venv-env:

- *Run command:* `pip install playwright`
- Then, after complete from previous step *run command:* `playwright install`

To check the playwright version, from cmd: `playwright --version`

To use sync playwright:

```
from playwright.sync_api import sync_playwright
```

Note 1: we can use sync playwright with: *with-keyword*.

Note 2: by default playwright run in *headless mode*

To use sync playwright with with-keyword:

```
# Same as: playwright = sync_playwright().start()
with sync_playwright() as playwright:
# This Is For Starting sync playwright Using with-context
    # This Will Start Chromium Browser With Headless-Option AS False
    # And The Slow Mode Will Increase x1000
    browser = playwright.chromium.launch(headless=False, slow_mo=1000)

    # This Will Create New Page Using The Browser
    page = browser.new_page()
    ||
    ||

    # This Will Close The Page
    page.close()

    # This Will Close The Browser
    browser.close()

    # Here AS We Use with-Keyword, Then stop()-Method Will Call Automatically
    # playwright.stop()
*****
```

To make page go to specific url:

```
page.goto("https://playwright.dev/python")
*****
```

To select element by its role, in playwright:

Note: Here we must set name-attr.

```
# This Will Return The Links With Text=Docs
docs_links = page.get_by_role('link', name='Docs')
*****
```

To click selected element, that is selected using get_by_role

```
# This Will Fire The Click-Event
docs_links.click()
*****
```

To get the url from the page-object:

```
print("The Current URL Is: ", page.url)
*****
```

To get element (text area, input, link) by text:

Note 1: In this way, any element who contains *full text/partial of text* will selected

Note 2: it has attr, that is: *exact* when it is true, we must set *full text*

```
text_element = page.get_by_text("with faded secondary text")
text_element.scroll_into_view_if_needed()
text_element.highlight()

middle_element = page.get_by_text("Middle")
middle_element.scroll_into_view_if_needed()
middle_element.highlight()
*****

text_element = page.get_by_text("attr", exact=True)
print("The element with exact is, display: ", text_element.is_visible())
text_element.scroll_into_view_if_needed()
text_element.highlight()
*****
```

```
password_input = page.locator("#floatingPassword")
password_input.scroll_into_view_if_needed()
password_input.highlight()

email_input_by_placeholder = page.get_by_placeholder("name@example.com")
email_input_by_placeholder.scroll_into_view_if_needed()
email_input_by_placeholder.highlight()
*****
```

To get images by alt-text:

```
from playwright.sync_api import sync_playwright
import time

with sync_playwright() as playwright:
    browser = playwright.chromium.launch(headless=False, slow_mo=1000)
    page = browser.new_page()

    page.goto("https://www.pexels.com/")
    page.get_by_alt_text("Nature in Motion Challenge").highlight()
    time.sleep(3)
    page.close()
    browser.close()
*****
```

```
playwright._impl._errors.Error: Locator.scroll_into_view_if_needed: Error: strict mode violation: get_by_title("Source Title") resolved to 3 elements:
  1) <cite title="Source Title">Source Title</cite> aka get_by_text("Source Title").first
  2) <cite title="Source Title">Source Title</cite> aka get_by_text("Source Title").nth(1)
  3) <cite title="Source Title">Source Title</cite> aka get_by_text("Source Title").nth(2)

Call log:
  - waiting for get_by_title("Source Title")
```

```
page.get_by_title("attribute").scroll_into_view_if_needed()
page.get_by_title("attribute").highlight()

page.get_by_title("Source Title").first.scroll_into_view_if_needed()
page.get_by_title("Source Title").first.highlight()
*****
```

If we want to specify the type of locator:

```
page.locator("css=h1").first.highlight()
*****
```

We have many pseudo-classes like: `:text('text here or part of it')`, `:text-is('exact matching')`, `:nth-match('matcher-here', #number of matcher like 1, 2, 3)`

From deepseek:

```
// Select the 1st button with exact text (handles whitespace)
```

```
const button = await page.locator('xpath=("//button[normalize-space(text())="Click Me"])[1]');
```

```
// Select the 3rd button containing "Save"
```

```
const button = await page.locator('xpath=("//button[contains(text(), "Save")])[3]');
```

```
// Or using nth
```

```
const button = await page.locator('//button[contains(text(), "Save")]').nth(2);
```

Click or dblclick or hover:

```
from playwright.sync_api import sync_playwright
```

```
import time
```

```
with sync_playwright() as playwright:
```

```
    browser = playwright.chromium.launch(headless=False, slow_mo=1000)
```

```
    page = browser.new_page()
```

```
    page.goto('https://bootswatch.com/flatly/')
```

```
    xpath_locator = "//a[@id='themes']"
```

```
    t1 = page.locator(f"xpath={xpath_locator}")
```

```
    t1.click()
```

```
    # In milli-seconds
```

```
    t1.dblclick(delay=1000)
```

```
    t1.click()
```

```
    # click the right button of mouse
```

```
    # t1.click(button='right')
```

```
    t2 = page.locator("css=button.btn.btn-outline-primary")
```

```
    t2.scroll_into_view_if_needed()
```

```
    t2.hover()
```

```
    time.sleep(2)
```

To fill the input field or type with delay:

```
from playwright.sync_api import sync_playwright
import time

with sync_playwright() as playwright:
    browser = playwright.chromium.launch(headless=False, slow_mo=1000)
    page = browser.new_page()
    page.goto('https://bootswatch.com/flatly/')
    t1 = page.get_by_placeholder("Enter email")
    t1.scroll_into_view_if_needed()
    # t1.fill("test@test.com")
    # Delay in milliseconds
    t1.type("test@test.com", delay=100)

    time.sleep(2)
```

Note: For radio btn && checkboxes && switches we have:

- Method 1: *check()*-method
- Method 2: *uncheck()*-method
- Method 3: *set_checked()*-method
- Method 4: *click()*-method

For select list (Single OR Multi):

- Method: *select_option(["option-here"])* -- for single list
- Method: *select_option(["option-01-here", "option-02-here", ...etc])* -- for multi select

Note: if we set option that doesn't exist on list then *Timeout-Exception* will be thrown

To upload file/s to page using playwright's element:

- For Single file upload: `element.set_input_files("destination-here-relative-or-absolute")`
 - For multiple file upload: `element.set_input_files(["file-01", "file-02", "file-03", ...etc])`
-

OR, we can use `page.expect_file_chooser` using with-keyword

Note: page is the page from `browser.new_page()`

Ex: with `page.expect_file_chooser()` as `file_elem`:

Ex: `element.click()`

Ex (outside the `with-context`): `file_chooser = file_elem.value`

Ex: `file_chooser.set_files(file-path-relative-or-absolute)` or
`file_chooser.set_files([...])`

Then, the result that `file_elem` contains the data of the dialog that appear for upload files.

To handle key pressed from keyboard in playwright:

- We can use: `element.press("KeyJ")`
 - *Note 1*: this will only send **J**
 - *Note 2*: we can repeat this step as we want
- To send shift with key using press: `element.press("Shift+KeyD")`
- We have also plus to *Key*, *Shift*:
 - **Control**
 - **ArrowLeft**
 - **Note**: when we set multiple keys to send we must separate between them by +

Note: when we select element using playwright and make action with it, the playwright background job check if:

1. The element loaded with page and *attached to dom*
2. The element is *visible*
3. The element is *enabled*
4. The element not *move, rotate, resize, ...etc* by the animation
5. Can *receive the event pointers* like (*click, dblclick, hover, focus* if not disabled, ...etc)

Note: if we select element that is *not appear in the page*, but it is inside the *dom*, then *Timeout error* will be thrown

Note: the *click()*-method has also *timeout attr*

We can change the waiting timeout for any action using *timeout='...'* *attribute* like the click-method

We can force the events to trigger if it can by using: *force=True attribute*

Note: if we use *force=True* then it will throw exception that element not visible *if not appear on the page*

To change the page load events, we can change the *wait_until attr* of *page.goto-method*

The locator object or first object has *wait_for()* or *wait_for(timeout=...)* method that are using with Ajax websites

Ex: *ele = page.locator('...').first*

Ex: *ele.wait_for()* # or we can use *wait_for(timeout=20_000)*

The *wait_for*-method also has *state-variable*

Also, the *page-object* has method called: *wait_for_selector('selector-here')*

The page object of browser object has event listener method, that is *on(...)-method* && *once(...)-method*:

Note: we have many events

Ex: `page.on('load', func_here)`

Ex: the `func_here` receive parameter that is event, in our example it is: `page`

Ex: `def on_load(page):`

Ex: `print('page is loaded with data: ', page)`

We can use event listeners to handle dialogues

Ex: `page.on('dialog', on_dialog)`

Ex: `def on_dialog(dialog):`

Ex: `dialog.accept()` or `dialog.dismiss()` and for prompt dialog we can use: `dialog.accept('text-here-for-testing')`

To download file in playwright we use with context:

Ex: `with page.expect_download() as t1:`

Ex: `download_btn.click()`

Ex(*outside the with*): `t2 = t1.value`

Ex(save to any directory because *when the main with finished the file will removed*): `t2.save_as('directory-here')`

We can also use events for download: `page.on('download', on_download)`, the `on_download` receive download-param that has also `save_as('directory-here')`-method

To save the state of our work we can use: `browser.new_context()` where browser is from `playwright.firefox.launch(headless=False, slow_mo=1000)`

If our test require manual interaction then we can pause the execution: `page.pause()`

To save the state of our context: `context.storage_state(path='path-here')`

If we want to reuse the state of context that we create: `browser.new_context(storage_state='path-here')`

From qwen.ai to locate all tr-elements of table:

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:

    browser = p.chromium.launch(headless=False) # Set headless=True if you don't need a GUI

    page = browser.new_page()

    page.goto("https://example.com") # Replace with your target URL

    # Locate all <tr> elements inside a <table>

    tr_elements = page.locator("table tr") # or "table >> tr"

    # Get count of rows

    count = tr_elements.count() # here we can use tr_elements.all()-method

    print(f"Found {count} <tr> elements.")

    # Optionally, extract text from each row

    for i in range(count):

        text = tr_elements.nth(i).text_content()

        print(f"Row {i + 1}: {text.strip()}")

    browser.close()
```
